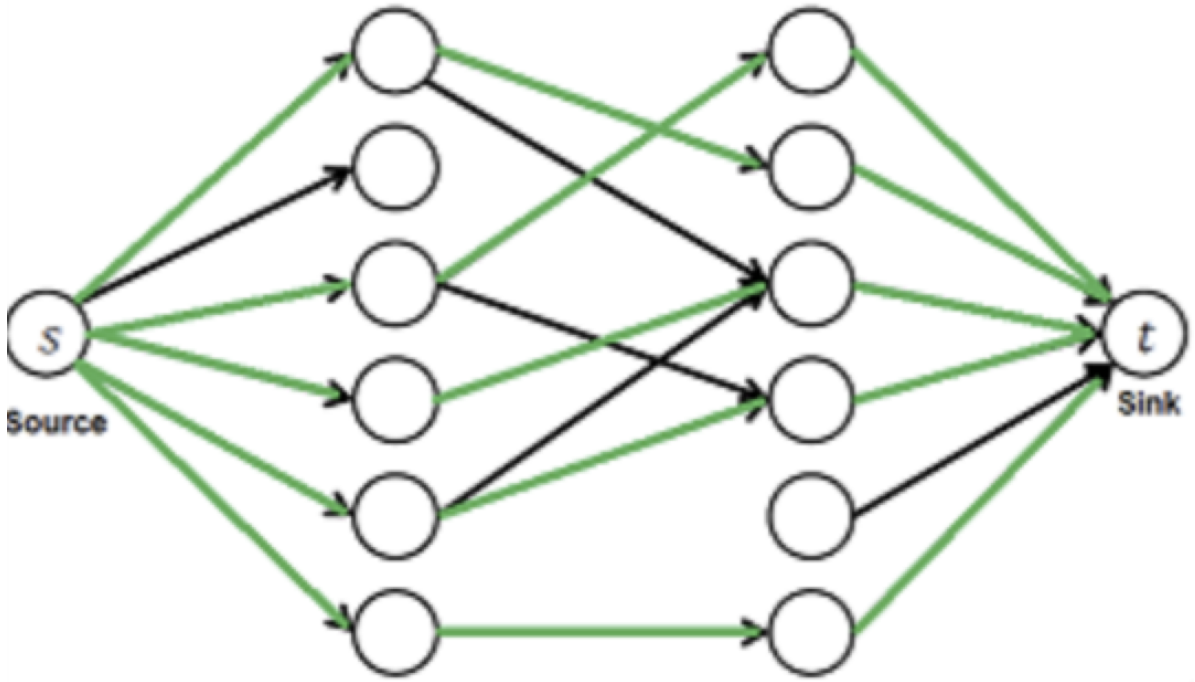
Bipartite Matching Problem

Problem:

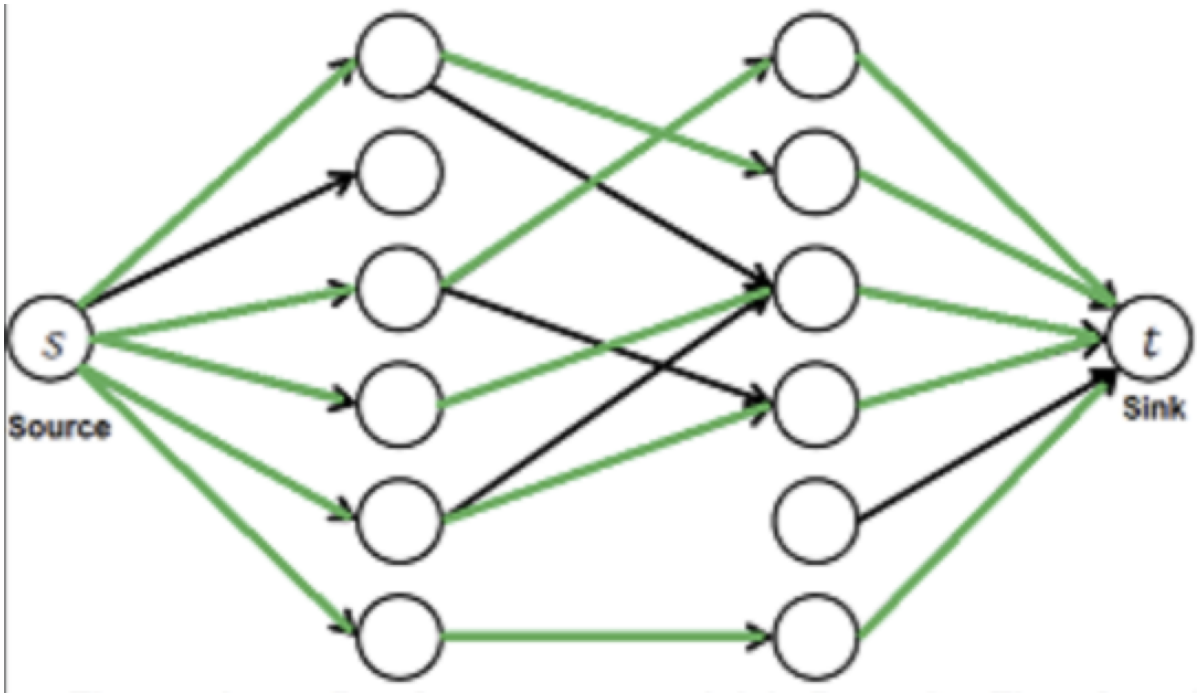
- Given a bipartite graph $G = (L \cup R, E)$, find the **maximum matching**.
- A matching $M \subseteq E$ is a set of edges such that no two edges share a vertex.



Applications: Job assignment, Student-course allocation, Network

Reduction to Maximum Flow

- Add a source s and connect it to all vertices in the left partition L.
- Add a sink t and connect it to all vertices in the right partition R.
- Assign capacity 1 to all edges.
- Solve the maximum flow problem from s to t.



Constructing the Flow Graph

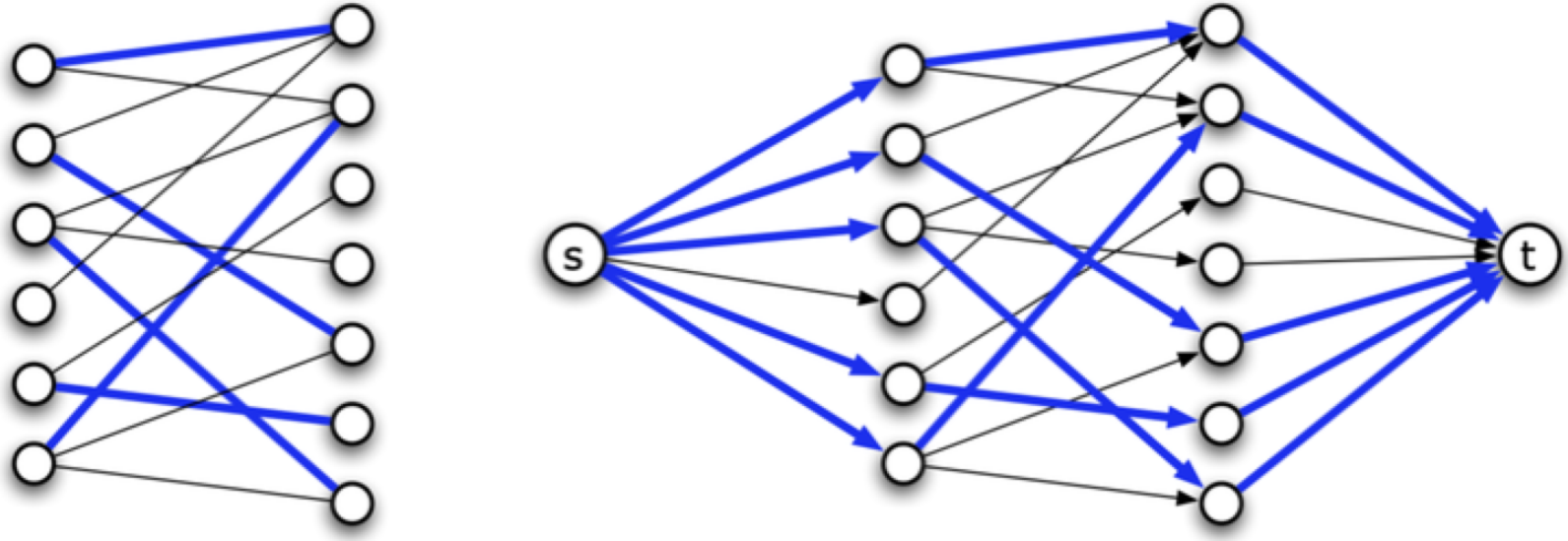
Graph Construction:

- Input graph: $G = (L \cup R, E)$.
- Augmented graph $G' = (V', E')$:
  - $V' = L \cup R \cup \{s, t\}$.
  - E' includes:
    - (s, u) for $u \in L$,
    - (v, t) for $v \in R$,
    - (u, v) for $(u, v) \in E$.
- Capacities: 1 for all edges.

Correctness of the Reduction

Why Does This Work?

- Any flow corresponds to a matching:
  - Capacity 1 ensures no two edges in M share a vertex.
- Any matching corresponds to a flow:
  - Construct a flow where $f(u, v) = 1$ for edges in M.
- Maximum flow ensures maximum matching.



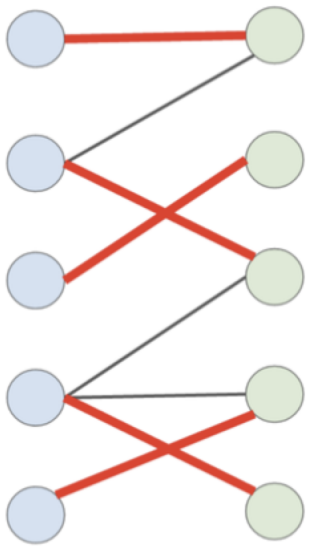
Perfect Bipartite Matching

Definition:

- A matching $M \subseteq E$ is **perfect** if every node in $X \cup Y$ appears in exactly one edge of M.

Necessary Conditions:

- $|X| = |Y|$: Equal sizes of partitions.
- For any subset $S \subseteq X$, $|N(S)| \geq |S|$: Every subset S of X has enough neighbors in Y.



Capacity Setup for Hall's Marriage Theorem

Graph Construction: To apply the Max-Flow Min-Cut Theorem, we modify the bipartite graph $G = (X \cup Y, E)$ as follows:

- Add a source node s and edges (s, u) for all $u \in X$ with capacity 1.
- Add a sink node t and edges (v, t) for all $v \in Y$ with capacity 1.
- Assign infinite capacity (∞) to edges between X and Y in G.

Hall's Marriage Theorem

Theorem

If $|N(S)| \geq |S|$ for all subsets $S \subseteq X$, then there is a perfect matching.

- By Max-Flow Min-Cut Theorem:

$$\text{Max flow } = \text{Min cut } .$$

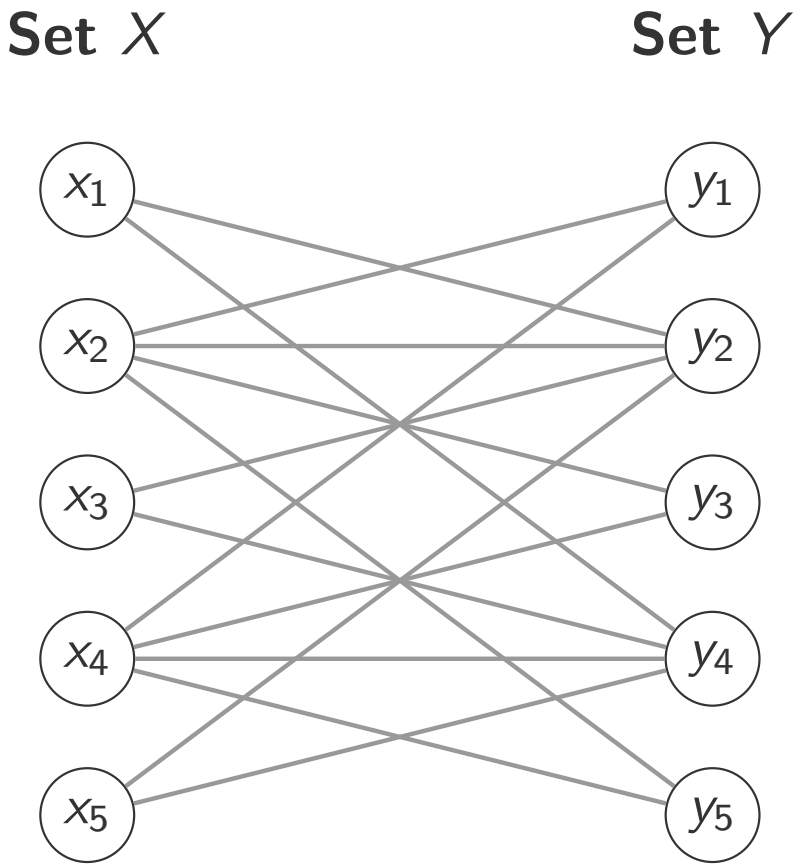
- The min-cut is $s \cup S_1 \cup S_2$, where $S_1 \subseteq X$, $S_2 \subseteq Y$.
- Let $T_1 = X - S_1$, $T_2 = Y - S_2$.
- Since $s \cup S_1 \cup S_2$ is a cut, there are no edges from S_1 to T_2, so the maximum flow is $|T_1| + |S_2|$.

$$|T_1| + |S_2| < |X| \iff |S_2| < |S_1| \implies N(S_1) < |S_1|$$

Conclusion: The min-cut size equals $|X|$ if and only if Hall's condition holds, proving the theorem.

Example

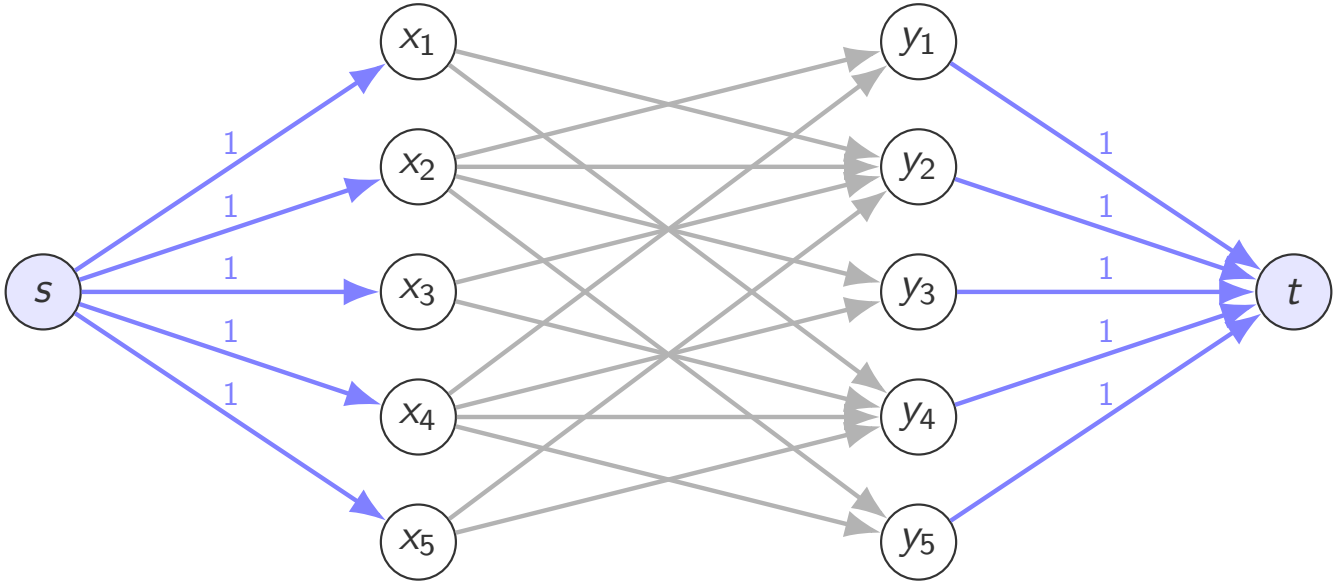
Consider $G = (X, Y, E)$ with $|X| = |Y| = 5$.



Formulating the Flow Network

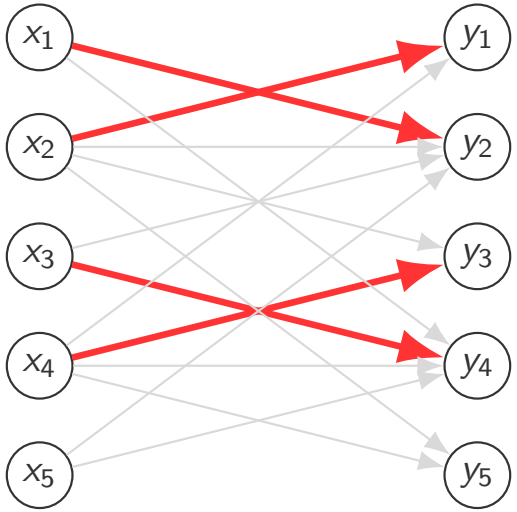
To rigorously evaluate the existence of a matching, we map the bipartite graph to a directed flow network $G' = (V', E')$.

- Add a source s and a sink t.
- Directed edges (s, x_i) with capacity $c(s, x_i) = 1$.
- Directed edges (y_j, t) with capacity $c(y_j, t) = 1$.
- Edges $(x_i, y_j) \in E$ are directed with capacity $c(x_i, y_j) = \infty$.



Maximum Flow

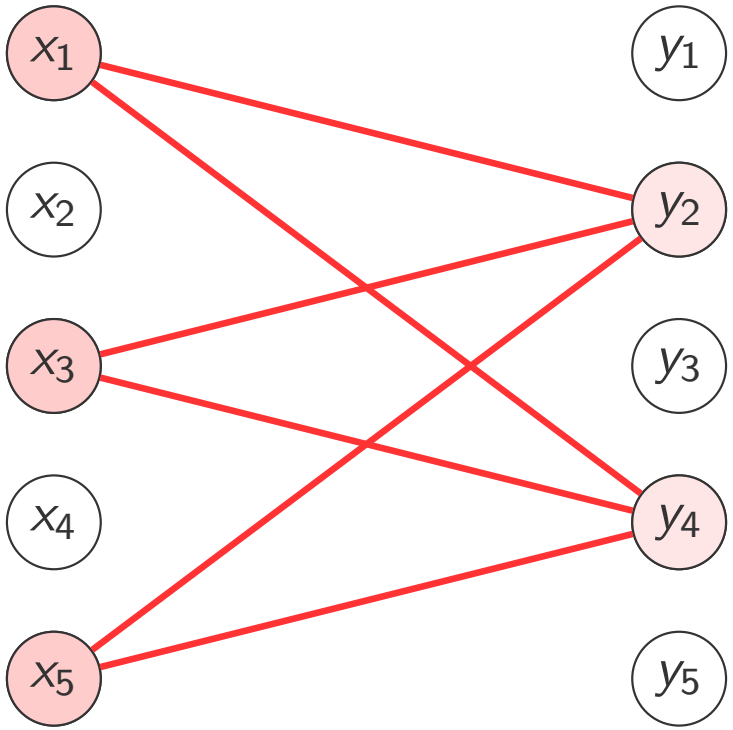
We push flow through augmenting paths until no $s \to t$ path exists in the residual graph. $s \to x_2 \to y_1 \to t$ (Flow $+1$), $s \to x_4 \to y_3 \to t$ (Flow $+1$)m $s \to x_1 \to y_2 \to t$ (Flow $+1$), $s \to x_3 \to y_4 \to t$ (Flow $+1$). The maximum flow saturates at **4**. Since $4 < |X|$, no perfect matching exists. Node x_5 is blocked because its only neighbors (y_2, y_4) are saturated.



Constructing the Minimum Cut

By the Max-Flow Min-Cut theorem, the capacity of the minimum cut (V_s, V_t) is exactly 4. We determine V_s by tracing reachable nodes from s in the **residual graph**.

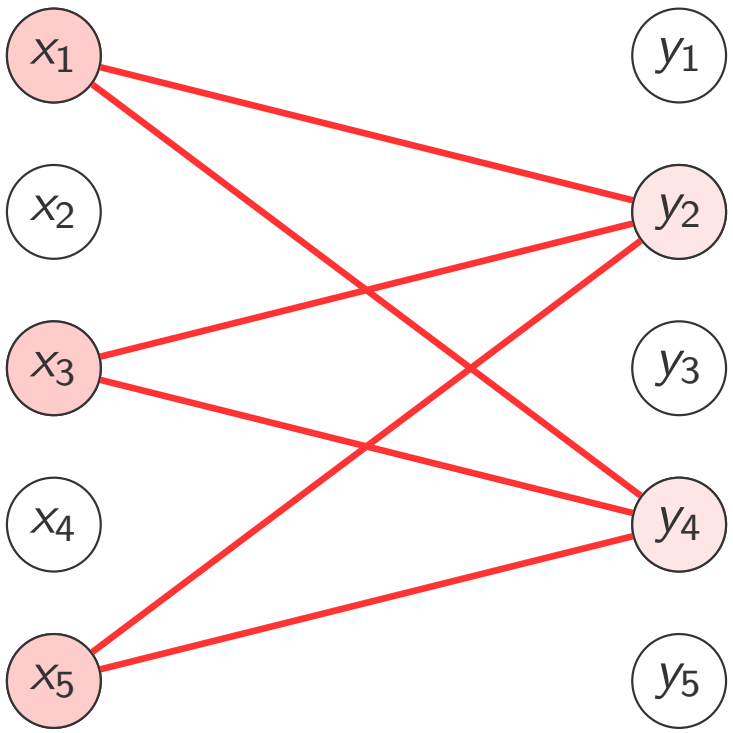
- $V_s = \{s, x_1, x_3, x_5, y_2, y_4\}$
- $V_t = \{x_2, x_4, y_1, y_3, y_5, t\}$



Isolating the Violating Subset S

We extract the problematic subset directly from the source-side of the minimum cut.

- Define $S = X \cap V_s = \{x_1, x_3, x_5\}$.
- The neighborhood of S is restricted to V_s, thus $N(S) = Y \cap V_s = \{y_2, y_4\}$.



Problem Description: Exam Scheduling

Inputs:

- c: Courses, where course i has $E[i]$ enrolled students.
- r: Rooms, where room j has $S[j]$ seats.
- t: Time-slots $f_1, \ldots, f_t$.
- p: Proctors, each with availability $T[k] \subseteq \{f_1, \ldots, f_t\}$.

Constraints:

1. $E[i] \leq S[j]$: A course can only be assigned to a room with enough seats.
2. Each room can host at most one exam per time-slot.
3. A proctor k can only attend exams in available time-slots $T[k]$.
4. Each proctor can attend at most 3 exams.

Goal: Find a schedule of c tuples (course, room, time, proctor) satisfying all constraints, or determine that no valid schedule exists.

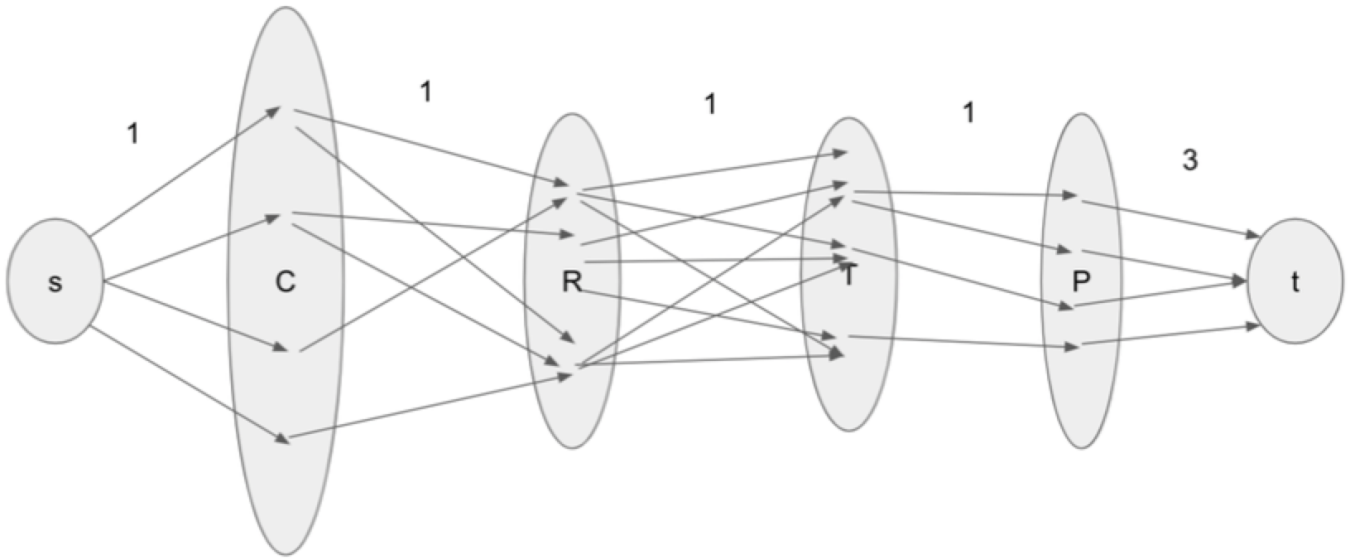
Network Flow Formulation

The graph consists of vertices for courses (C), rooms (R), time-slots (T), proctors (P), and source/sink nodes (s, t).

Edges:

- $s \to C[i]$: Capacity 1 (one exam per course).
- $C[i] \to R[j]$: Capacity 1 if $E[i] \leq S[j]$ (room can fit students).
- $R[j] \to T[\ell]$: Capacity 1 (room used for one exam per time-slot).
- $T[\ell] \to P[k]$: Capacity 1 if $\ell \in T[k]$ (proctor available at time).
- $P[k] \to t$: Capacity 3 (proctor can oversee at most 3 exams).

Objective: Find a maximum flow f from s to t.



Reduction to Scheduling

Algorithm:

1. Construct the network $G = (V, E)$ as described.
2. Compute the maximum flow f from s to t.
3. Extract flow paths of the form:

$$(s, C[i], R[j], T[\ell], P[k], t).$$

4. Return course$_i$, room$_j$, time$_\ell$, proctor$_k$) for each path.
5. If the number of paths is less than c, output "No valid schedule."

Properties of the Solution:

- Each course is scheduled in one room and one time-slot.
- Each room is used for only one exam per time-slot.
- Each proctor is assigned at most 3 exams and only during their available slots.

Proof of Correctness: Valid Schedule

Valid Schedule:

- Each course $C[i]$ is assigned to a valid room $R[j]$ where $E[i] \leq S[j]$, by edge construction.
- Each room $R[j]$ is assigned to at most one exam per time-slot, as edges from $R[j]$ to $T[\ell]$ have capacity 1.
- Each proctor $P[k]$ attends at most 3 exams and only during their available time-slots $T[k]$, as:
  - Edges from $T[\ell]$ to $P[k]$ are restricted by $T[k]$.
  - Edges from $P[k]$ to t have capacity 3.

Conclusion: The returned tuples form a valid schedule satisfying all constraints.

Proof of Correctness: Maximum Size

Maximum Size:

- Any flow f corresponds to a schedule with the same number of tuples as the flow value:

$$f(s, C[i]) = 1 \implies C[i] \text{ is scheduled.}$$

- Conversely, any valid schedule corresponds to a feasible flow f in G:
  - Assign $f(s, C[i]) = 1$, $f(C[i], R[j]) = 1$, etc., for each tuple (course, room, time, proctor) .
- By Max-Flow Min-Cut Theorem, the maximum flow corresponds to the largest number of tuples, ensuring the schedule is optimal.

Conclusion: If value of max flow $= c$, we have a valid schedule for all courses. Otherwise, no such schedule exists.

Hardness and Complexity

- **Algorithms:** Precise, step-by-step procedures to solve problems.
- **Complexity:** Measures the "resources" (e.g., time, space, randomness) needed to solve problems.
- **Key Questions:**
  - What does it mean to "solve" a problem?
  - Can a problem be solved?
  - How "hard" is it to solve a problem?

Models of Computation

What is our model of computation ("computer", the computational setup)?

- **RAM Model:** Assumes constant-time basic operations.
- **Turing Machines:** Abstract infinite tape and read/write head, and finite control..
- **Quantum Computing:** Explores superposition and entanglement.
- **Parallel/Distributed Models:** Multi-processor and communication models.
- **Communication Complexity:** Minimizing inter-party communication.
- **Boolean Circuits:** Logic gates and hardware modeling.
- **Streaming Models:** Single-pass, limited memory analysis.
- **Interactive Proofs:** Verifier-prover interaction frameworks.
- **Space-Bounded Models:** Memory-constrained computations.
- **Decision Trees:** Binary trees for query and decision problems.

The Space of Problems and Complexity

- **What Can Be Solved?**
  - Which problems are computable? What defines "hard" problems?
  - Explore time, space, randomness, and approximations.
- **Relationships Between Problems:**
  - Reductions: Showing one problem's complexity via another.
  - Hierarchies: $P \subseteq NP \subseteq PSPACE \subseteq EXP$.
  - Open questions like $P = NP$?.
- **Tradeoffs in Complexity:**
  - Time vs. space, exact vs. approximate solutions.
  - Power of randomness: BPP, RP.
- **Proof and Verification:**
  - Verifying vs. solving: P vs. NP, interactive proofs, zero-knowledge.
  - Complexity of certifying solutions.

The Halting Problem

To formalize the limits of computation, we define the Halting Problem as a decision problem.

Halting Problem

Given a source code string $\langle A \rangle$ representing an algorithm, and an input string x, determine whether the computation $A(x)$ halts in a finite number of steps.

Theorem: The Halting Problem is undecidable. There exists no algorithm that correctly decides the Halting Problem for all inputs in finite time.

Proof Strategy: We proceed by contradiction via a diagonalization argument. We will assume a universal decider exists and construct a reduction that forces this decider into a logical impossibility.

The Hypothesis and Construction

The Assumption: Assume for the sake of contradiction that there exists an algorithm H that decides the Halting Problem.

$$H(\langle A \rangle, x) = \begin{cases} 1 & \text{if } A(x) \text{ halts} \\ 0 & \text{if } A(x) \text{ does not halt} \end{cases}$$

The Diagonalization Machine: Using H as a subroutine, we construct a new algorithm D

- **Self-Query:** D evaluates $H(\langle X \rangle, \langle X \rangle)$ to predict if X halts on its own code.
- **Adversarial Loop:** If H outputs 1 (predicts X halts), D loops forever.
- **Adversarial Halt:** If H outputs 0 (predicts X loops), D immediately halts.

The Contradiction

Proof of Contradiction: Consider the evaluation of D on its own source code, $\langle D \rangle$.

1. **Case 1:** $D(\langle D \rangle)$ **halts.** By definition of our decider, $H(\langle D \rangle, \langle D \rangle)$ must evaluate to 1. Looking at the construction of D, if H returns 1, D intentionally enters an infinite loop. This contradicts the premise that $D(\langle D \rangle)$ halts.
2. **Case 2:** $D(\langle D \rangle)$ **does not halt.** By definition of our decider, $H(\langle D \rangle, \langle D \rangle)$ must evaluate to 0. Looking at the construction of D, if H returns 0, D immediately halts and returns. This contradicts the premise that $D(\langle D \rangle)$ does not halt.

Both logical branches yield a contradiction. Therefore, our initial assumption must be false. The decider H cannot exist, proving that the Halting Problem is undecidable.

What is a Polynomial-Time Algorithm?

- An algorithm is **polynomial-time (poly-time)** if its runtime is $O(N^k)$, where:
  - N: Number of bits to represent the input.
  - k: A constant ($k \geq 0$).
- Poly-time algorithms are considered **efficient**.
- Examples of poly-time algorithms:
  - Sorting: Insertion Sort ($O(n^2)$), Merge Sort ($O(n \log n)$).
  - Graph algorithms: Connectivity, MST, Shortest Path ($O(m \log n)$).
  - Dynamic Programming: Longest Increasing Subsequence ($O(n^2)$).
- Non-poly-time examples:
  - Fibonacci numbers ($O(n)$, input size $O(\log n)$).
  - Knapsack ($O(nW)$, input size $O(n \log W)$).

Decision Problems

- A **decision problem** has a binary answer: Yes/No, True/False, or 1/0.
- Many practical problems are decision problems:
  - Is element e in array A? (Binary Search)
  - Is graph G connected? (DFS/BFS)
  - Does graph G have a Hamiltonian cycle?
  - Is graph G 3-colorable?
- Non-decision problems can often be transformed into decision problems:
  - Example: "Find the MST of graph G" becomes "Is the MST weight $> K$?"

Verifiers and Polynomial-Time Verification

- A **verifier** checks whether a proposed solution (proof y) to a problem Π is correct:
  - Input: Problem instance x and proof y.
  - Output: True if y proves $\Pi(x) = 1$, otherwise False.
- Verifiers focus on **verification**, not finding solutions.
- Examples:
  - **Graph Connectivity:** Proof is a spanning tree.
  - **Hamiltonian Cycle:** Proof is the Hamiltonian cycle.
  - **3-Coloring:** Proof is a valid 3-coloring of the graph.
- **Efficient Verifiers:** Run in polynomial time relative to the input and proof size.

Class P: Problems Solvable in Polynomial Time

- Class P consists of all decision problems that can be solved by a polynomial-time algorithm.
- Examples:
  - Sorting: Merge Sort ($O(n \log n)$).
  - Graph Problems: Connectivity, MST, Shortest Paths.
  - Dynamic Programming: LIS, DAG Longest Path.

Class NP: Problems Verifiable in Polynomial Time

- Class NP consists of decision problems that can be verified by a polynomial-time verifier.
- Examples:
  - **Hamiltonian Cycle:** Given a cycle, verify if it visits all vertices.
  - **3-Coloring:** Given a coloring, verify no edge is monochromatic.
  - **Knapsack:** Given a subset, verify it satisfies constraints.
- **Key Observation:** $P \subseteq NP$.
- Reason: If a problem is solvable in poly-time (P), the solution itself can act as the proof for verification (NP).

Are all problems verifiable?

Example: Language L of graphs **without** a Hamiltonian cycle.

- **Easy to verify G has a Hamiltonian cycle:** Show the cycle.
- **Hard to verify G has no Hamiltonian cycle:** Requires checking all possible cycles.

Asymmetry:

- If $x \in L$: Provide a certificate y where $A(x, y) = \text{TRUE}$.
- If $x \notin L$: Must verify $A(x, y) = \text{FALSE}$ for all y (exponential).

Takeaway: Verifiability in NP is one-sided—proving $x \notin L$ is often infeasible.

The Big Question: Is $P = NP$?

- The central open question in theoretical computer science:

  Is every problem in NP also in P?

- Equivalent Question: Can every efficiently verifiable problem be efficiently solved?
- Current Status:
  - Most researchers believe $P \neq NP$.
  - No formal proof either way.
- Implications:
  - If $P = NP$: Many "hard" problems become solvable efficiently.
  - If $P \neq NP$: Some problems in NP remain inherently intractable.

P vs NP

- Complexity classes P and NP formalize the notions of efficient solvability and verifiability.
- Most practical problems belong to NP, but only some are in P.
- The $P = NP$ question is a profound challenge in computer science with deep implications.

What are NP-Hard Problems?

- A problem Π is called **NP-hard** if:
  - Every problem in **NP** can be reduced to Π in polynomial time.
- In other words, if Π can be solved in polynomial time, then:

$$\mathbf{P} = \mathbf{NP}.$$

- NP-hard problems may or may not be in NP:
  - They are not necessarily decision problems.
  - They may be optimization or search problems.
- **Examples** of NP-hard problems:
  - *Traveling Salesperson Problem (TSP)* — optimization version.
  - *3-SAT (Search Version)* — finding a satisfying assignment.

What are NP-Complete Problems?

- A decision problem Π is **NP-complete** if:
  1. Π is NP-hard.
  2. Π belongs to the class NP (has a poly-time verifier).
- Intuitively:
  - NP-complete problems are the hardest problems in NP.
  - Solving any NP-complete problem efficiently solves all problems in NP.

Why Study NP-Hard and NP-Complete Problems?

- **If $P = NP$:**
  - A poly-time algorithm for one NP-hard problem solves all problems in NP.
  - Impacts cryptography, optimization, and theoretical foundations.
- **If $P \neq NP$:**
  - Proving NP-hardness justifies the lack of efficient algorithms.
  - Focus shifts to approximations and heuristics.

Proving NP-Hardness via Reductions

- To prove a problem A is NP-hard:
  1. Identify a known NP-hard problem B.
  2. Reduce B to A in polynomial time.
  3. If A can be solved in polynomial time, B can also be solved in polynomial time, implying $P = NP$.
- **Important Notes:**
  - Reductions must be computable in polynomial time.
  - You don't need a poly-time algorithm for A; the reduction suffices.

Proving NP-Completeness

- To prove A is NP-complete:
  1. Show A is NP-hard via reduction.
  2. Prove $A \in NP$ by designing a polynomial-time verifier.
- **Example Problems:**
  - Hamiltonian Cycle, 3-SAT, Vertex Cover, Traveling Salesperson (decision version), etc.

Example of a Reduction

- **Problem A: Vertex Cover**
  - Input: Graph $G = (V, E)$, integer k
  - Question: Is there a set of $\leq k$ vertices that touches every edge?
- **Problem B: Hitting Set**
  - Input: Universe U, collection of sets $\mathcal{S} \subseteq 2^U$, integer k
  - Question: Is there a subset $H \subseteq U$, $|H| \leq k$, that intersects every set in $\mathcal{S}$?
- **Reduction Idea:**
  - Let $U = V$ (vertices of the graph)
  - For each edge $(u, v) \in E$, create the set $\{u, v\} \in \mathcal{S}$
  - Then a hitting set $H \subseteq U$ of size $\leq k$ covers all edges

Why Study NP-Hardness and NP-Completeness?

- **If $P = NP$:**
  - A polynomial-time algorithm for any NP-complete problem solves all problems in NP.
  - Significant real-world implications for cryptography, optimization, and more.
- **If $P \neq NP$:**
  - Proving NP-hardness justifies the absence of efficient algorithms for these problems.
  - Focus shifts to approximations, heuristics, and alternative models of computation.

Circuit-SAT: The First NP-Complete Problem

Problem Statement:

- **Input:** A Boolean circuit C with n input bits.
- The circuit uses logic gates: AND, OR, and NOT.
- **Question:** Is there an input $x \in \{0,1\}^n$ such that $C(x) = 1$?

Circuit-SAT is in NP

Why Circuit-SAT is in NP:

- A satisfying assignment $x \in \{0,1\}^n$ is a **witness**.
- We can verify the output $C(x)$ in polynomial time:
  - Evaluate the circuit gate by gate.
  - Use a topological order of the circuit (no cycles).
  - Total time: $O(\#$ of gates$)$.
- If $C(x) = 1$, then accept; otherwise, reject.

Circuit-SAT is NP-Hard

Cook-Levin Theorem:

- Every problem in *NP* can be reduced to Circuit-SAT in polynomial time.
- Idea: Simulate the verifier Turing machine as a Boolean circuit.
- The circuit checks if a given witness causes the machine to accept.
- This shows Circuit-SAT is at least as hard as any problem in NP.

Conclusion: Circuit-SAT is NP-complete.